

Test Case Design Techniques

Definition:

Test Case Design Techniques are methods used by testers to design effective test cases and identify defects by testing different types of inputs, conditions, and scenarios.

1. Equivalence Partitioning (EP)

Definition:

Equivalence Partitioning is a technique where we divide the input data into **valid and invalid groups** and test one or more values from each group.

Example:

Suppose an **Age** field accepts values from **18 to 60**.

We can divide the data into 3 groups:

Group	Range	Example
Invalid	Below 18	17
Valid	18–60	30
Invalid	Above 60	61

Test Cases:

- Enter **17** → ❌ Invalid
- Enter **30** → ✅ Valid
- Enter **61** → ❌ Invalid

Simple meaning:

Divide the data into groups and test a value from each group.

2. Boundary Value Analysis (BVA)

Definition:

Boundary Value Analysis is a technique where we test the values **at and around the minimum and maximum boundaries**.

Example:

Suppose an **Age** field accepts **18 to 60**.

We test:

Test Value	Expected Result
17	✗ Invalid
18	✓ Valid
19	✓ Valid
59	✓ Valid
60	✓ Valid
61	✗ Invalid

Values to remember:

Minimum - 1, Minimum, Minimum + 1, Maximum - 1, Maximum, Maximum + 1

For 18–60:

17, 18, 19, 59, 60, 61

Simple meaning:

Test the values at and around the boundaries.

3. Decision Table Testing

Definition:

Decision Table Testing is used when the system's output depends on **multiple conditions**.

Example:

Suppose a user can log in only when:

- Username is valid
- Password is valid

Username	Password	Expected Result
Valid	Valid	✓ Login successful

Username	Password	Expected Result
Valid	Invalid	✗ Login failed
Invalid	Valid	✗ Login failed
Invalid	Invalid	✗ Login failed

Here, we are testing different **combinations of conditions**.

Simple meaning:

When multiple conditions affect the result, use a Decision Table.

4. State Transition Testing

Definition:

State Transition Testing is used to test how the application behaves when it moves from **one state to another based on an action or event**.

Example:

Suppose a bank account becomes locked after **3 incorrect password attempts**.

Current State	Action	Next State
Active	Correct password	Logged in
Active	Wrong password	Login failed
Login failed	Wrong password again	Login failed
Login failed	3rd wrong attempt	Account locked
Account locked	Correct password	Account remains locked

Simple meaning:

Test how the system changes from one state to another based on an action.

Another simple example:

Order:

Order Placed → Order Confirmed → Shipped → Delivered

We test whether the application correctly moves from one state to the next.

5. Use Case Testing

Definition:

Use Case Testing is used to design test cases based on **real user activities or business scenarios**.

Example:

Suppose we have an **Online Shopping Application**.

A user performs:

Login → Search Product → Select Product → Add to Cart → Make Payment → Order Placed

We create test cases for this complete user flow.

Example Test Cases:

1. Verify user can log in.
2. Verify user can search for a product.
3. Verify user can add the product to the cart.
4. Verify user can make payment.
5. Verify order is placed successfully.

Simple meaning:

Test the application based on real-world user workflows.

6. Error Guessing

Definition:

Error Guessing is a technique where testers use their **experience and knowledge** to predict where defects are likely to occur.

It is not based on a fixed formula.

Example:

For a Login page, a tester may try:

- Username left blank
- Password left blank
- Both fields blank
- Invalid username

- Invalid password
- Special characters
- Very long username
- Very long password
- Spaces in the fields
- Multiple login attempts

These are common areas where defects may occur.

Simple meaning:

Use testing experience to guess where bugs might occur and test those areas.

7. Pairwise Testing

Definition:

Pairwise Testing is used when there are **many combinations of input values**. Instead of testing every possible combination, we make sure important **pairs of values** are tested.

Example:

Suppose we have:

Browser:

- Chrome
- Firefox
- Edge

Operating System:

- Windows
- Mac

Language:

- English
- Hindi

Testing every possible combination would require:

$$3 \times 2 \times 2 = \mathbf{12 \text{ combinations}}$$

Pairwise testing tries to cover the important **pairs of values** with fewer test cases.

Simple meaning:

Test important pairs of input values instead of testing every possible combination.

Quick Revision

Technique	What does it test?	Simple Example
Equivalence Partitioning	Groups of data	Valid / Invalid age groups
Boundary Value Analysis	Boundary values	17, 18, 19, 59, 60, 61
Decision Table	Multiple conditions	Username + Password
State Transition	Change from one state to another	Active → Locked
Use Case Testing	Real user workflow	Login → Order → Payment
Error Guessing	Possible defects based on experience	Blank fields, special characters
Pairwise Testing	Combinations/pairs of inputs	Browser + OS combinations

Easy Way to Remember

EP → Groups

BVA → Boundaries

Decision Table → Conditions

State Transition → States

Use Case → User Flow

Error Guessing → Experience

Pairwise → Combinations